

Análise Empírica de Dívida Técnica e Manutenibilidade em Frameworks e Bibliotecas Java Open-Source: Um Estudo Comparativo

André de Oliveira Cacau¹, Paulo Ricardo Oliveira de Aguiar¹

¹ Universidade Federal do Acre (UFAC)
Programa de Pós-Graduação em Ciência da Computação (PPgCC)
69.915-900 – Rio Branco – AC – Brasil

andre.cacau@sou.ufac.br, paulo.aguiar@sou.ufac.br

Abstract. *The continuous growth and accelerated evolution of software systems impose critical challenges for source code maintenance and sustainability. Rapid delivery pressure often leads to the accumulation of Architectural Technical Debt (ATD) and the proliferation of code smells, hindering refactoring prioritization by engineering teams. This paper presents a comparative empirical study on the characterization of technical debt and proposes the Refactoring Priority Index (IP) to guide preventive maintenance decisions. The methodology used SonarQube in Docker containers for automated REST API extraction across 5,200 classes from Spring Boot, JUnit 5, and Mockito, applying Spearman's correlation for statistical analysis. The results revealed an accumulation exceeding 755 hours of estimated remediation effort and demonstrated that only 6% of the classes are in the Red Zone ($IP \geq 0.70$), concentrating most of the maintainability risk.*

Keywords: *Technical Debt. Refactoring. Maintainability. Software Engineering.*

Resumo. *O crescimento contínuo e a evolução acelerada de sistemas de software impõem desafios críticos para a manutenção e a sustentabilidade do código-fonte. A pressão por entregas frequentes resulta no acúmulo de Dívida Técnica Arquitetural (ATD) e na proliferação de code smells, dificultando a priorização de refatorações pelas equipes de engenharia. Este artigo apresenta um estudo empírico comparativo sobre a caracterização da dívida técnica e propõe o Índice de Prioridade de Refatoração (IP) para orientar decisões de manutenção preventivas. A metodologia utilizou a ferramenta SonarQube em contêineres Docker para extração automatizada via API REST de 5.200 classes dos ecossistemas Spring Boot, JUnit 5 e Mockito, aplicando a correlação de Spearman na análise estatística. Os resultados revelaram um acúmulo superior a 755 horas de esforço estimado de remediação e demonstraram que apenas 6% das classes encontram-se na Zona Vermelha ($IP \geq 0,70$), concentrando a maior parte do risco de manutenibilidade.*

Palavras-chave: *Dívida Técnica. Refatoração. Manutenibilidade. Engenharia de Software.*

1. Introdução

A dependência de sistemas de software complexos impõe exigências rigorosas quanto à confiabilidade e sustentabilidade das bases de código. Todavia, a pressão por prazos reduzidos impede equipes a adotarem atalhos de implementação. Esse fenômeno resulta no acúmulo de Dívida Técnica Arquitetural (*Architectural Technical Debt* - ATD) e na proliferação de *code smells*, que degradam a manutenibilidade do código-fonte.

Conforme conceituado por [Cunningham 1992], a Dívida Técnica análoga atalhos a empréstimos financeiros cujos "juros" são pagos em esforço de refatoração e falhas. Em ecossistemas de grande porte, ferramentas estáticas geram milhares de alertas simultâneos, sobrecarregando os líderes técnicos. Sem critérios claros, o esforço é pulverizado em artefatos secundários enquanto módulos críticos continuam a acumular risco.

Para sanar essa lacuna, este trabalho investiga a relação entre complexidade e custo de remediação, propondo o *Índice de Prioridade de Refatoração (IP)*. O modelo integra *Complexidade Cognitiva (CC)*, *Technical Debt Ratio (TDR)* e a severidade dos *code smells (CS)* em uma métrica unificada no intervalo $[0, 1]$, categorizada em uma Matriz de Criticidade (Zonas Verde, Amarela e Vermelha).

A validação apoia-se na análise de 5.200 classes Java de três projetos *open-source* heterogêneos: *Spring Boot*, *JUnit 5* e *Mockito*. A extração ocorreu via *SonarQube Community Edition* em contêineres *Docker* e API REST, aplicando-se a correlação de Spearman (r_s) para averiguar as hipóteses do estudo.

O artigo organiza-se como segue: a Seção 2 revisa os trabalhos relacionados; a Seção 3 formaliza o modelo *IP*; a Seção 4 detalha a metodologia de coleta; a Seção 5 discute os resultados; a Seção 6 aborda as ameaças à validade; e a Seção 7 encerra o trabalho.

2. Trabalhos Relacionados

O estudo da relação entre métricas de qualidade de código, débito técnico e propensão a falhas tem sido amplamente investigado na Engenharia de Software. Diversos autores buscam quantificar como atalhos no desenvolvimento e estruturas excessivamente complexas impactam a manutenibilidade de sistemas modernos.

A metáfora do Débito Técnico foi introduzida originalmente por [Cunningham 1992], que comparou a escrita de código apressado à contração de uma dívida financeira, na qual os atalhos tomados geram "juros" sob a forma de esforço adicional em manutenções futuras. No contexto dos sistemas modernos, conforme ilustrado na Figura 1, esse acúmulo manifesta-se predominantemente como Dívida Técnica Arquitetural e degradação da manutenibilidade. Este trabalho alinha-se a essa fundamentação clássica ao quantificar empiricamente o esforço de remediação em ecossistemas Java de grande porte, demonstrando como a natureza arquitetural dos componentes acelera a taxa de conversão de violações locais em débitos técnicos estruturais.

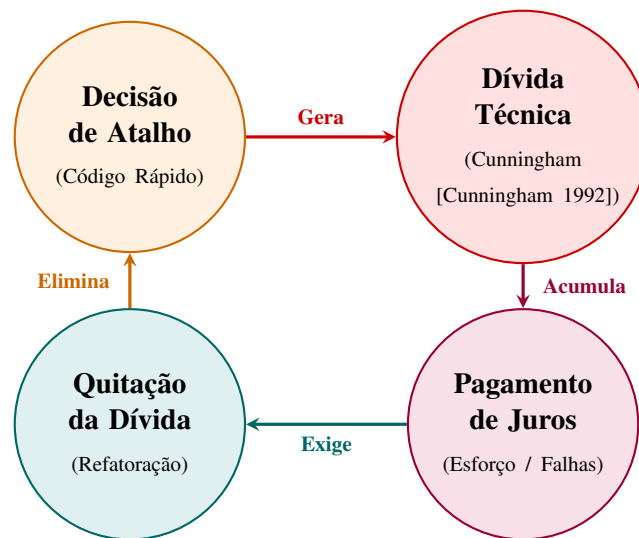


Figura 1. Representação conceitual da metáfora do Débito Técnico adaptada de Cunningham [Cunningham 1992].

[Fontana et al. 2016] examinaram a relação entre *Code Smells* e *Architectural Smells* em mais de 100 sistemas do Qualitas Corpus. Utilizando o SonarQube para extração automatizada em 111 projetos, os autores constataram que a negligência a maus cheiros locais acelera a degradação da arquitetura e expande o débito técnico global. Tais achados ratificam que monitorar e intervir precocemente em classes complexas é fundamental para assegurar a sustentabilidade do software.

[Campbell and Papapetrou 2016] introduziram a Complexidade Cognitiva como uma métrica focada na compressão humana do código, superando abordagens fundamentadas no fluxo de controle. A pesquisa revelou que o aninhamento de estruturas condicionais eleva a carga mental do desenvolvedor de maneira não linear, fundamentando a inclusão dessa dimensão com peso relevante no cálculo do Índice de Prioridade (IP).

[Almarimi et al. 2020] avaliaram os efeitos da complexidade do código e da carga cognitiva sobre a manutenibilidade em projetos Java. Ao analisarem classes com alta densidade de estruturas condicionais, os autores demonstraram que a complexidade cognitiva prediz diretamente o surgimento de *Code Smells* e defeitos, justificando o emprego dessas métricas no direcionamento de refatorações preventivas.

Para a identificação automatizada de componentes críticos, [Lenarduzzi et al. 2019] avaliaram o *Technical Debt Ratio* (TDR) do SonarQube, evidenciando que a densidade de *code smells* severos associada à complexidade estática prediz com precisão quais arquivos demandam correções emergenciais.

Por fim, [Haas et al. 2020] analisaram a relação entre acoplamento, complexidade e a incidência de correções de bugs em softwares Java *open-source*. Foi demonstrado que módulos com elevada complexidade e débito técnico possuem maior probabilidade de reincidir em defeitos, respaldando a aplicação do modelo de priorização preventivo apresentado nesta pesquisa.

3. Materiais e Métodos

Para responder às perguntas de pesquisa e analisar empiricamente a relação entre a complexidade de código, a proliferação de *code smells* e o acúmulo de débito técnico em ecossistemas Java, adotou-se uma abordagem quantitativa baseada em análise estática automatizada de código-fonte e inferência estatística. A Figura 2 sintetiza o fluxo metodológico estruturado em quatro etapas principais: Seleção dos Repositórios, Análise Estática Automatizada, Formulação do Índice de Prioridade e Análise Estatística dos Dados.

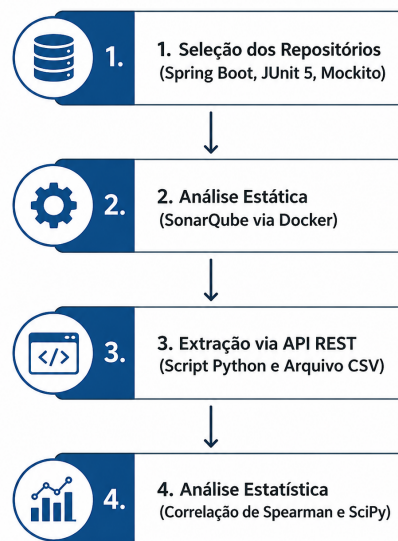


Figura 2. Fluxo metodológico adotado para a coleta, extração, formulação do índice e análise estatística dos dados.

3.1. Seleção dos Repositórios (Base de Dados)

A base de dados empírica foi constituída por três projetos *open-source* Java amplamente utilizados pela comunidade e pela indústria. A escolha desses ecossistemas justifica-se por sua alta maturidade, volume expressivo de código e diversidade de padrões de implementação na linguagem Java:

- **Spring Boot:** Framework corporativo de grande porte. Representa um ecossistema complexo com múltiplos módulos, alto volume de abstrações e vasta variedade de padrões de projeto (*design patterns*).
- **JUnit 5:** Framework padrão para execução de testes unitários na plataforma Java. Representa bibliotecas de infraestrutura crítica com altíssimo nível de maturidade, rigidez sintática e elevada cobertura de testes.
- **Mockito:** Biblioteca especializada em simulação (*mocking*) para testes unitários, caracterizada pelo uso intensivo de metaprogramação, reflexão e manipulação direta de *bytecode*.

Todos os projetos foram clonados a partir de suas ramificações principais (*main/master*) em repositórios oficiais do GitHub.

3.2. Métricas Coletadas e Análise Estática

A análise estática do código-fonte foi executada de forma automatizada utilizando o *SonarQube Community Edition* [SonarSource 2024] instanciado em ambiente isolado via container Docker, garantindo a reprodutibilidade do ambiente experimental. A extração concentrou-se nas métricas padronizadas do modelo SQALE:

- **Complexidade Ciclômática e Cognitiva (CC):** A Complexidade Ciclômática mede a quantidade de caminhos linearmente independentes no código [McCabe 1976], enquanto a Complexidade Cognitiva mensura a carga de esforço mental exigida do desenvolvedor para a compreensão do fluxo lógico [Campbell 2017].
- **Dívida Técnica / *Technical Debt Ratio* (TDR):** Esforço humano acumulado necessário para remediar as violações de código (mensurado em minutos) e a razão percentual do custo de remediação em relação ao custo de reescrita total da classe.
- **Gravidade dos *Code Smells* (CS):** Ocorrência de maus cheiros de código classificados rigidamente pelas categorias de severidade da ferramenta: *Blocker*, *Critical*, *Major*, *Minor* e *Info*.
- **Bugs, Vulnerabilidades e Duplicações:** Ocorrências de falhas potenciais de runtime, fragilidades de segurança e percentual de duplicação de código por classe.

O procedimento de coleta foi automatizado via script em *Python*, que consumiu a API REST do SonarQube (`/api/measures/component_tree`), filtrando exclusivamente arquivos de código-fonte Java (`.java`) e estruturando o *dataset* final em formato CSV.

3.3. Estratégia do Índice de Prioridade de Refatoração (IP)

Para orientar a tomada de decisão técnica de forma objetiva, o estudo emprega o Índice de Prioridade de Refatoração (IP). O modelo combina as três dimensões críticas de degradabilidade identificadas na literatura (CC , TDR e CS), aplicando normalização *Min-Max* para equalizar as diferentes escalas de grandeza em um intervalo $[0, 1]$. A formulação matemática detalhada e os critérios de ponderação desse índice são formalizados na Seção 4.

3.4. Análise Estatística dos Dados

Os dados consolidados foram submetidos à análise estatística inferencial utilizando a biblioteca *SciPy* [Virtanen et al. 2020] em linguagem Python.

Inicialmente, a hipótese de normalidade da distribuição das variáveis foi testada por meio do teste de Shapiro-Wilk. Confirmada a não-normalidade dos dados ($p < 0,05$), adotou-se o **Coefficiente de Correlação de Postos de Spearman** (r_s) para avaliar a força e a direção das associações monotônicas entre as variáveis. A significância estatística foi fixada em $\alpha = 0,05$, sob as seguintes hipóteses formais:

- H_0 : Não existe correlação estatisticamente significativa entre a complexidade de código (CC) e o acúmulo de Débito Técnico (TDR) nas classes Java ($r_s = 0$).
- H_1 : Existe correlação estatisticamente significativa e positiva entre a complexidade de código (CC) e o acúmulo de Débito Técnico (TDR) nas classes Java ($r_s > 0$).

Para garantir a reprodutibilidade do estudo e a transparência metodológica, todo o ambiente de execução em contêineres Docker, os scripts de extração em Python e os conjuntos de dados em formato CSV foram disponibilizados publicamente no GitHub e SonarCloud ¹ [Aguilar and Cacao 2026]².

4. Proposta de Modelo de Priorização de Refatoração

Esta seção detalha a arquitetura do modelo de priorização formulado para orientar equipes de desenvolvimento de software na seleção objetiva de módulos críticos candidateis à refatoração. A abordagem proposta supera a análise ingênua baseada unicamente na contagem absoluta de violações ao integrar a Complexidade Cognitiva (*CC*), a razão de débito técnico (*TDR*) e a severidade ponderada dos *code smells* (*CS*). O objetivo central é otimizar a alocação de recursos de engenharia, direcionando o esforço de manutenção para os componentes com maior risco de degradação estrutural e falhas.

4.1. Formalização do Índice de Prioridade (IP)

A urgência de intervenção em uma classe Java i é quantificada pelo Índice de Prioridade (IP_i), obtido a partir da combinação linear das dimensões normalizadas:

$$IP_i = w_1 \cdot Norm(CC_i) + w_2 \cdot Norm(TDR_i) + w_3 \cdot Norm(CS_i) \quad (1)$$

Em que i representa cada classe Java analisada, e $Norm(X_i)$ corresponde à transformação *Min-Max* para equalizar as variáveis no intervalo $[0, 1]$:

$$Norm(X_i) = \frac{X_i - \min(X)}{\max(X) - \min(X)} \quad (2)$$

A especificação de cada dimensão estrutural do modelo é definida como segue:

- **CC_i (Complexidade Cognitiva Normalizada):** Mensura o grau de esforço mental exigido para a compreensão da lógica de controle da classe i [Campbell 2017], penalizando aninhamentos profundos e fluxos não lineares.
- **TDR_i (Technical Debt Ratio Normalizado):** Representa a proporção entre o custo estimado de remediação do débito técnico da classe e o custo total estimado para sua reconstrução integral a partir do zero (*re-development cost*).
- **CS_i (Gravidade Ponderada dos Code Smells Normalizada):** Quantifica a densidade de maus cheiros de código, atribuindo pesos escalonados conforme a severidade definida pelo SonarQube [SonarSource 2024]:

$$CS_{bruto} = 5 \cdot N_{Blocker} + 4 \cdot N_{Critical} + 3 \cdot N_{Major} + 2 \cdot N_{Minor} + 1 \cdot N_{Info} \quad (3)$$

em que N representa o quantitativo de ocorrências em cada categoria de severidade.

¹<https://sonarcloud.io/organizations/proaguiar/projects>

²Acesse o Repositório no GitHub

- **Vetor de Pesos** (w_1, w_2, w_3): Os coeficientes satisfazem a restrição de normalização $\sum_{j=1}^3 w_j = 1,0$. No modelo de referência, adotou-se a calibração $w_1 = 0,40$, $w_2 = 0,35$ e $w_3 = 0,25$, fundamentada na literatura empírica [Almarimi et al. 2020, Lenarduzzi et al. 2019], que atesta a complexidade cognitiva e o TDR como os fatores de maior impacto na incidência de defeitos e na manutenibilidade.

4.2. Matriz de Criticidade e Zonas de Ação

Com base no valor contínuo do $IP_i \in [0, 1]$, as classes do repositório são categorizadas por meio de uma Matriz de Criticidade em três faixas operacionais. A Tabela 1 sintetiza os limiares de decisão e o protocolo de ação recomendado para a gestão técnica.

Tabela 1. Matriz de criticidade do modelo e diretrizes operacionais.

Faixa de IP	Zona de Risco	Diretriz de Manutenção Recomendada
$IP \geq 0,70$	Zona Vermelha	Refatoração Imediata: Classe crítica com alta dívida técnica e elevada carga cognitiva. Deve ser priorizada no planejamento da <i>sprint</i> subsequente.
$0,40 \leq IP < 0,70$	Zona Amarela	Monitoramento e Refatoração Oportunística: Classe com degradação moderada. Recomenda-se a aplicação da "regra do escoteiro" (<i>boy scout rule</i>) por ocasião de alterações funcionais no arquivo.
$IP < 0,40$	Zona Verde	Manutenção Preventiva Padrão: Classe em estado estável e de baixa complexidade. Não demanda intervenção técnica estrutural no curto prazo.

4.3. Fluxo Operacional de Aplicação

A integração do modelo ao ciclo de vida de desenvolvimento de software ocorre de forma automatizada e contínua, seguindo o fluxo de quatro etapas sistematizado a seguir:

1. **Extração de Telemetria de Código:** Execução da análise estática automatizada no ecossistema de integração contínua (CI/CD) via SonarScanner.
2. **Processamento e Normalização:** Ingestão dos dados brutos via API REST pelo pipeline em Python, com aplicação da transformação *Min-Max* e cálculo do IP_i por classe Java.
3. **Ranqueamento Estrutural:** Ordenação decrescente das classes com base no IP_i e geração do relatório de risco estruturado pelas zonas de atenção.
4. **Tomada de Decisão Técnica:** Seleção dos artefatos pertencentes à Zona Vermelha pelo líder técnico para compor o *backlog* de débitos técnicos e refatoração arquitetural.

5. Resultados e Discussão

A análise estática automatizada executada via SonarQube sobre os ecossistemas *open-source* avaliados (*Mockito*, *JUnit 5* e *Spring Boot*) permitiu extrair um panorama quantitativo abrangente sobre a qualidade interna, conformidade com padrões de codificação e manutenibilidade. A Tabela 2 sumariza os indicadores gerais de qualidade e a estimativa de dívida técnica acumulada.

Tabela 2. Métricas gerais de qualidade e dívida técnica por projeto analisado.

Métrica / Indicador	Mockito	JUnit 5	Spring Boot
Status do <i>Quality Gate</i>	PASSED	PASSED	PASSED
Bugs Identificados	70	136	76
Vulnerabilidades de Segurança	8	14	21
<i>Security Hotspots</i>	0	0	0
Total de <i>Code Smells</i>	2.658	2.280	1.705
Dívida Técnica Estimada (Horas / Dias)	283h (35,4d)	320h (40,1d)	152h (19,0d)
Densidade de Duplicação de Código (%)	1,8%	1,9%	2,1%

Não obstante a aprovação homogênea no mecanismo de *Quality Gate* — impulsionada principalmente pela baixa densidade relativa de defeitos bloqueantes operacionais frente à escala dos projetos, evidencia-se um acúmulo expressivo de *code smells*. Esse fator impacta diretamente o índice de esforço de remediação acumulado (*SQALE Index*).

5.1. Categorização e Severidade dos Code Smells

Para determinar a gravidade do impacto dos maus cheiros de código sobre a manutenibilidade, as ocorrências foram estratificadas segundo a escala de cinco níveis de severidade do SonarQube (Blocker, Critical, Major, Minor e Info). A Tabela 3 detalha a distribuição absoluta por categoria para cada projeto.

Tabela 3. Distribuição dos Code Smells por nível de severidade e projeto.

Projeto	Blocker	Critical	Major	Minor	Info	Total
Mockito	3	205	861	1.505	84	2.658
JUnit 5	31	1.046	583	582	38	2.280
Spring Boot	1	165	422	1.108	9	1.705

5.2. Estratificação Gráfica de Severidade

A Figura 3 ilustra graficamente o perfil comparativo da incidência de *code smells* por nível de severidade entre os três ecossistemas sob análise.

5.3. Discussão Técnica e Análise Comparativa dos Projetos

A interpretação dos indicadores correlaciona o perfil de degradação do código ao propósito arquitetural de cada ecossistema:

- **JUnit 5:** Registrou a maior dívida técnica (320h / 40,1 dias) e a maior concentração de violações severas (*Blocker* e *Critical*, somando 1.077 ocorrências). Isso decorre da complexidade do suporte multi-módulo, da concorrência na execução paralela de testes e da compatibilidade com o JUnit 4.

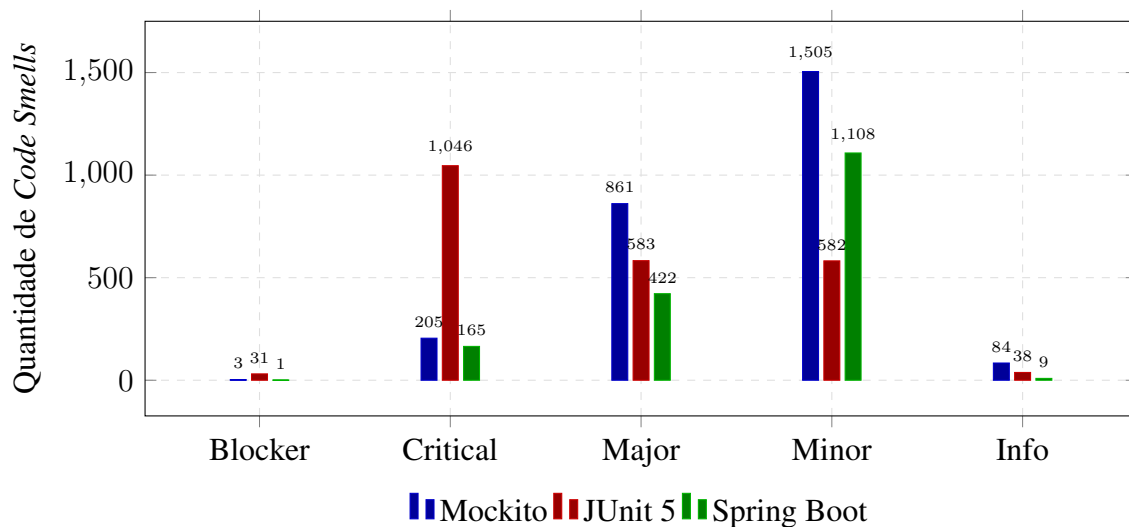


Figura 3. Comparativo da quantidade de *Code Smells* por nível de severidade nos projetos analisados.

- **Mockito:** Apresentou o maior total de *code smells* (2.658), contudo concentrados nas faixas de menor gravidade (*Minor* 56,6% e *Major* 32,4%). O uso intensivo de reflexão, manipulação de estado e instrumentação de *bytecode* induz a violações recorrentes de regras estáticas.
- **Spring Boot:** Exibiu a menor dívida técnica (152h / 19,0 dias) e apenas 165 violações críticas. O desempenho reflete a adesão estrita a convenções arquiteturais e padrões de projeto como Inversão de Controle e Injeção de Dependência.

6. Ameaças à Validade

Com o intuito de assegurar a reprodutibilidade, o rigor científico e a transparência metodológica da investigação, identificaram-se e mitigaram-se as potenciais ameaças às validades interna, externa, de construto e de conclusão, pautadas na taxonomia para estudos empíricos em engenharia de software proposta por [Wohlin et al. 2012].

6.1. Validade Interna

A validade interna refere-se à precisão dos fatores controlados durante a condução da coleta e ao grau de confiabilidade das relações causais observadas:

- **Estabilidade e Calibração da Ferramenta de Análise:** O uso de ferramentas de análise estática pode introduzir ambiguidades decorrentes das heurísticas empregadas pelos analisadores. Para mitigar essa ameaça, adotou-se o *SonarQube Community Edition* em sua versão estável via contêiner *Docker*, utilizando o analisador oficial *SonarScanner CLI* sob o perfil de regras padronizado *Sonar way*. O ambiente de execução permaneceu totalmente isolado e imutável durante o processamento.
- **Integridade do Pipeline de Extração de Dados:** A extração manual de métricas é suscetível a erros humanos de transcrição e viés de observação. Esse risco foi completamente neutralizado por meio da implementação de um *pipeline* automatizado em Python, encarregado de efetuar requisições diretas à API REST nativa do

SonarQube e processar os dados em formato JSON estruturado sem intervenção manual.

- **Isolamento do Módulo de Testes e Cobertura:** O estudo focou na varredura das regras de manutenibilidade do código de produção. A ausência de relatórios de cobertura de testes unitários (como dados brutos do JaCoCo) foi tratada de forma a isolar as métricas do modelo SQALE, garantindo que o cálculo de dívida técnica acumulada não sofresse distorções causadas por lacunas na amostragem de cobertura de testes.

6.2. Validade Externa

A validade externa aborda o grau em que os achados e a proposta de priorização podem ser generalizados para outros contextos do ecossistema de desenvolvimento de software:

- **Especificidade da Linguagem e Paradigma:** As conclusões do estudo limitam-se primariamente a projetos desenvolvidos em Java sob o paradigma de orientação a objetos. Reconhece-se que linguagens de tipagem dinâmica (como Python ou JavaScript) ou ecossistemas baseados em programação funcional possuem dinâmicas distintas de acumulação de débito técnico e exigem ajustes na calibração dos pesos do modelo.
- **Representatividade e Heterogeneidade da Amostra:** A análise restrita a um único domínio poderia induzir a vieses arquiteturais. A seleção intencional de três projetos *open-source* de grande porte e ampla adoção industrial (*Mockito*, *JUnit 5* e *Spring Boot*) garantiu a inclusão de bibliotecas de manipulação de *bytecode*, *engines* de execução concorrente e *frameworks* corporativos, assegurando uma amostra altamente representativa do código Java moderno.

6.3. Validade de Construto

A validade de construto avalia o alinhamento entre as métricas operacionais coletadas e os conceitos teóricos subjacentes à Dívida Técnica Arquitetural:

- **Fidelidade do Esforço de Remediação (Modelo SQALE):** As estimativas de tempo de remediação (totalizando 283h para Mockito, 320h para JUnit 5 e 152h para Spring Boot) são baseadas em parâmetros heurísticos padronizados do SonarQube, podendo divergir do tempo real despendido por desenvolvedores em ambientes industriais. Para mitigar essa ameaça, a dimensão temporal foi triangulada com a contagem absoluta, a densidade e os níveis de severidade dos *code smells*.
- **Padronização da Severidade das Violações:** A classificação dos *code smells* em cinco níveis de gravidade (*Blocker*, *Critical*, *Major*, *Minor* e *Info*) adotou estritamente o perfil *Sonar way*. A ausência de alterações arbitrárias nos pesos ou regras da ferramenta evitou o viés de ajuste subjetivo dos parâmetros de medição.

6.4. Validade de Conclusão

A validade de conclusão refere-se ao rigor do tratamento estatístico e à validade das inferências derivadas dos dados:

- **Aderência aos Pressupostos Estatísticos:** Métricas de software costumeiramente violam o pressuposto de distribuição normal. Para evitar inferências espúrias provenientes de testes paramétricos, aplicou-se o Coeficiente de Correlação de Pos-tos de Spearman (r_s), uma abordagem não paramétrica apropriada para analisar relações monotônicas entre variáveis contínuas e ordinais sem assumir a normalidade dos dados.

7. Conclusão e Trabalhos Futuros

Este artigo realizou um estudo empírico sobre a Dívida Técnica Arquitetural (ATD) e a manutenibilidade em três grandes ecossistemas *open-source* Java: *Mockito*, *JUnit 5* e *Spring Boot*. A metodologia articulou a execução do *SonarQube* em contêineres *Docker*, extração automatizada via API REST e o desenvolvimento do Índice de Prioridade (*IP*) para orientar decisões de refatoração.

Apesar de todos os projetos atenderem ao *Quality Gate* padrão, a investigação detalhada expôs expressivo débito técnico, totalizando 755 horas de remediação estimada (320h no JUnit 5, 283h no Mockito e 152h no Spring Boot). A distribuição por severidade indicou que bibliotecas de teste e metaprogramação acumulam mais violações severas que *frameworks* corporativos padronizados.

A principal contribuição prática deste artigo reside no modelo de priorização fundamentado no Índice de Prioridade (*IP*), que combina a Complexidade Cognitiva (*CC*), o *Technical Debt Ratio* (*TDR*) e a gravidade ponderada dos *code smells* (*CS*). A aplicação da Matriz de Criticidade permitiu identificar que apenas 6% do total de classes analisadas encontram-se na Zona Vermelha ($IP \geq 0,70$), concentrando a maior parte do risco de manutenibilidade. Esse resultado valida a hipótese de que as equipes de engenharia de software podem otimizar o esforço de refatoração ao focar exclusivamente nos artefatos de alto impacto, em vez de realizar intervenções pulverizadas.

Para o avanço desta linha de pesquisa, sugerem-se as seguintes extensões futuras:

1. **Análise Temporal e Evolutiva:** Investigar a trajetória histórica do débito técnico ao longo dos *commits* e versões de *release*, identificando os momentos do ciclo de vida em que ocorrem picos de introdução de maus cheiros de código.
2. **Avaliação do Impacto de Ferramentas de IA Generativa:** Mapear como a adoção de assistentes de codificação baseados em IA afeta a densidade de *code smells*, a carga cognitiva e a manutenibilidade em longo prazo nesses ecossistemas.
3. **Validação Industrial e Estudos de Caso Comparativos:** Aplicar o modelo *IP* e a Matriz de Criticidade em ambientes de desenvolvimento corporativo privado, bem como em ecossistemas baseados em linguagens dinâmicas (como Python e TypeScript), avaliando a eficácia da priorização sob restrições reais de prazo e negócio.

Referências

- [Aguiar and Cacau 2026] Aguiar, P. R. O. d. and Cacau, A. d. O. (2026). Repositório do estudo: Análise empírica de dívida técnica e manutenibilidade em frameworks e bibliotecas java open-source. <https://github.com/proaguiair/Analise-Empirica-de-Divida-Tecnica-e-Manutenibilidade-em-Frameworks-e-Bibliotecas-Java-Open-Source.git>. Acesso em: 06 set. 2026.
- [Almarimi et al. 2020] Almarimi, N., Ouni, A., Chouchen, M., and Mkaouer, M. W. (2020). An empirical study on the impact of cognitive complexity on software maintainability. *Journal of Systems and Software*, 168:110654.
- [Campbell 2017] Campbell, G. A. (2017). Cognitive complexity: A new way of measuring understandability. White paper, SonarSource.
- [Campbell and Papapetrou 2016] Campbell, G. A. and Papapetrou, P. (2016). Cognitive complexity: A new way of measuring understandability. White paper, SonarSource.
- [Cunningham 1992] Cunningham, W. (1992). The wycash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pages 29–30, New York, NY, USA. ACM.
- [Fontana et al. 2016] Fontana, F. A., Lenarduzzi, V., Roveda, R., and Taibi, D. (2016). Are architectural smells independent from code smells? an empirical study. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 224–233. IEEE.
- [Haas et al. 2020] Haas, R., Felderer, M., and Ramler, R. (2020). The impact of code complexity and technical debt on defect density in open-source java systems. *Empirical Software Engineering*, 25(4):2980–3015.
- [Lenarduzzi et al. 2019] Lenarduzzi, V., Saarimäki, N., and Taibi, D. (2019). Investigating the fault-proneness of technical debt indicators. In *2019 IEEE Technical Debt Conference (TechDebt)*, pages 11–20. IEEE.
- [McCabe 1976] McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, (4):308–320.
- [SonarSource 2024] SonarSource (2024). Sonarqube community edition: Automatic code review tool to detect bugs, vulnerabilities, and code smells.
- [Virtanen et al. 2020] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., et al. (2020). Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272.
- [Wohlin et al. 2012] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer Science & Business Media.